

Annex A

(informative)

Language syntax summary

A.1 Notation

The notation is described in 6.1.

A.2 Lexical grammar

A.2.1 Lexical elements

(6.4.1) *token*:

keyword
identifier
constant
string-literal
punctuator

(6.4.1) *preprocessing-token*:

header-name
identifier
pp-number
character-constant
string-literal
punctuator
each universal character name that cannot be one of the above
each non-white-space character that cannot be one of the above

A.2.2 Keywords

(6.4.2) *keyword*: one of

alignas	do	int	struct	while
alignof	double	long	switch	_Atomic
auto	else	nullptr	thread_local	_BitInt
bool	enum	register	true	_Complex
break	extern	restrict	typedef	_Decimal128
case	false	return	typeof	_Decimal32
char	float	short	typeof_unqual	_Decimal64
const	for	signed	union	_Generic
constexpr	goto	sizeof	unsigned	_Imaginary
continue	if	static	void	_Noreturn
default	inline	static_assert	volatile	

A.2.3 Identifiers

(6.4.3.1) *identifier*:

identifier-start
identifier *identifier-continue*

(6.4.3.1) *identifier-start*:

nondigit
XID_Start character
universal character name of class XID_Start

(6.4.3.1) *identifier-continue*:

digit
nondigit
XID_Continue character
universal character name of class XID_Continue

(6.4.3.1) *nondigit*: one of

–	a	b	c	d	e	f	g	h	i	j	k	l	m
	n	o	p	q	r	s	t	u	v	w	x	y	z
	A	B	C	D	E	F	G	H	I	J	K	L	M
	N	O	P	Q	R	S	T	U	V	W	X	Y	Z

(6.4.3.1) *digit*: one of

0	1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---	---

A.2.4 Universal character names

(6.4.4) *universal-character-name*:

\u	hex-quad
\U	hex-quad hex-quad

(6.4.4) *hex-quad*:

hexadecimal-digit	hexadecimal-digit	hexadecimal-digit	hexadecimal-digit
-------------------	-------------------	-------------------	-------------------

A.2.5 Constants

(6.4.5.1) *constant*:

integer-constant
floating-constant
enumeration-constant
character-constant
predefined-constant

(6.4.5.2) *integer-constant*:

decimal-constant	integer-suffix _{opt}
octal-constant	integer-suffix _{opt}
hexadecimal-constant	integer-suffix _{opt}
binary-constant	integer-suffix _{opt}

(6.4.5.2) *decimal-constant*:

nonzero-digit
decimal-constant ' _{opt} digit

(6.4.5.2) *octal-constant*:

0
octal-constant ' _{opt} octal-digit

(6.4.5.2) *hexadecimal-constant*:

hexadecimal-prefix hexadecimal-digit-sequence

(6.4.5.2) *binary-constant*:

binary-prefix binary-digit
binary-constant ' _{opt} binary-digit

(6.4.5.2) *hexadecimal-prefix*: one of

0x 0X

(6.4.5.2) *binary-prefix*: one of

0b 0B

(6.4.5.2) *nonzero-digit*: one of

1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---

(6.4.5.2) *octal-digit*: one of

0	1	2	3	4	5	6	7
---	---	---	---	---	---	---	---

hexadecimal-digit-sequence:

hexadecimal-digit
hexadecimal-digit-sequence ' _{opt} hexadecimal-digit

(6.4.5.2) *hexadecimal-digit*: one of

0 1 2 3 4 5 6 7 8 9
a b c d e f
A B C D E F

(6.4.5.2) *binary-digit*: one of

0 1

(6.4.5.2) *integer-suffix*:

unsigned-suffix long-suffix_{opt}
unsigned-suffix long-long-suffix
unsigned-suffix bit-precise-int-suffix
long-suffix unsigned-suffix_{opt}
long-long-suffix unsigned-suffix_{opt}
bit-precise-int-suffix unsigned-suffix_{opt}

(6.4.5.2) *bit-precise-int-suffix*: one of

wb WB

(6.4.5.2) *unsigned-suffix*: one of

u U

(6.4.5.2) *long-suffix*: one of

l L

(6.4.5.2) *long-long-suffix*: one of

ll LL

(6.4.5.3) *floating-constant*:

decimal-floating-constant
hexadecimal-floating-constant

(6.4.5.3) *decimal-floating-constant*:

fractional-constant exponent-part_{opt} floating-suffix_{opt}
digit-sequence exponent-part floating-suffix_{opt}

(6.4.5.3) *hexadecimal-floating-constant*:

hexadecimal-prefix hexadecimal-fractional-constant
binary-exponent-part floating-suffix_{opt}
hexadecimal-prefix hexadecimal-digit-sequence
binary-exponent-part floating-suffix_{opt}

(6.4.5.3) *fractional-constant*:

digit-sequence_{opt} . digit-sequence
digit-sequence .

(6.4.5.3) *exponent-part*:

e *sign_{opt} digit-sequence*
E *sign_{opt} digit-sequence*

(6.4.5.3) *sign*: one of

+ -

(6.4.5.3) *digit-sequence*:

digit
digit-sequence ' _{opt} digit

(6.4.5.3) *hexadecimal-fractional-constant*:

hexadecimal-digit-sequence_{opt} . hexadecimal-digit-sequence
hexadecimal-digit-sequence .

(6.4.5.3) *binary-exponent-part*:

p *sign_{opt} digit-sequence*
P *sign_{opt} digit-sequence*

- (6.4.5.3) *floating-suffix*: one of
f l F L df dd dl DF DD DL
- (6.4.5.4) *enumeration-constant*:
identifier
- (6.4.5.5) *character-constant*:
*encoding-prefix*_{opt} ' *c-char-sequence* '
- (6.4.5.5) *encoding-prefix*: one of
u8 u U L
- (6.4.5.5) *c-char-sequence*:
c-char
c-char-sequence c-char
- (6.4.5.5) *c-char*:
any member of the source character set except
the single-quote ' , backslash \ , or new-line character
escape-sequence
- (6.4.5.5) *escape-sequence*:
simple-escape-sequence
octal-escape-sequence
hexadecimal-escape-sequence
universal-character-name
- (6.4.5.5) *simple-escape-sequence*: one of
\ ' \ " \ ? \ \
\ a \ b \ f \ n \ r \ t \ v
- (6.4.5.5) *octal-escape-sequence*:
\ *octal-digit*
\ *octal-digit octal-digit*
\ *octal-digit octal-digit octal-digit*
- (6.4.5.5) *hexadecimal-escape-sequence*:
\ x *hexadecimal-digit*
hexadecimal-escape-sequence hexadecimal-digit
- (6.4.5.6) *predefined-constant*:
false
true
nullptr

A.2.6 String literals

- (6.4.6) *string-literal*:
*encoding-prefix*_{opt} " *s-char-sequence*_{opt} "
- (6.4.6) *s-char-sequence*:
s-char
s-char-sequence s-char
- (6.4.6) *s-char*:
any member of the source character set except
the double-quote " , backslash \ , or new-line character
escape-sequence

A.2.7 Punctuators

(6.4.7) *punctuator*: one of

[] () { } . ->
++ -- & * + - ~ !
/ % << >> < > <= >= == != ^ | && ||
? : :: ; ...
= *= /= %= += -= <=> >=> &= ^= |=
, # ##
<: :> <% %> %: %:~:

A.2.8 Header names

(6.4.8) *header-name*:

< *h-char-sequence* >
" *q-char-sequence* "

(6.4.8) *h-char-sequence*:

h-char
h-char-sequence *h-char*

(6.4.8) *h-char*:

any member of the source character set except
the new-line character and >

(6.4.8) *q-char-sequence*:

q-char
q-char-sequence *q-char*

(6.4.8) *q-char*:

any member of the source character set except
the new-line character and "

A.2.9 Preprocessing numbers

(6.4.9) *pp-number*:

digit
. *digit*
pp-number *identifier-continue*
pp-number ' *digit*
pp-number ' *nondigit*
pp-number **e** *sign*
pp-number **E** *sign*
pp-number **p** *sign*
pp-number **P** *sign*
pp-number .

A.3 Phrase structure grammar

A.3.1 Expressions

(6.5.2) *primary-expression*:

identifier
constant
string-literal
(*expression*)
generic-selection

(6.5.2.1) *generic-selection*:

_Generic (*assignment-expression* , *generic-assoc-list*)

(6.5.2.1) *generic-assoc-list*:

generic-association
generic-assoc-list , *generic-association*

(6.5.2.1) *generic-association*:

type-name : *assignment-expression*

default : *assignment-expression*

(6.5.3.1) *postfix-expression*:

primary-expression

postfix-expression [*expression*]

postfix-expression (*argument-expression-list*_{opt})

postfix-expression . *identifier*

postfix-expression -> *identifier*

postfix-expression ++

postfix-expression --

compound-literal

(6.5.3.1) *argument-expression-list*:

assignment-expression

argument-expression-list , *assignment-expression*

(6.5.3.6) *compound-literal*:

(*storage-class-specifiers*_{opt} *type-name*) *braced-initializer*

(6.5.3.6) *storage-class-specifiers*:

storage-class-specifier

storage-class-specifiers *storage-class-specifier*

(6.5.4.1) *unary-expression*:

postfix-expression

++ *unary-expression*

-- *unary-expression*

unary-operator *cast-expression*

sizeof *unary-expression*

sizeof (*type-name*)

alignof (*type-name*)

(6.5.4.1) *unary-operator*: one of

& * + - ~ !

(6.5.5) *cast-expression*:

unary-expression

(*type-name*) *cast-expression*

(6.5.6) *multiplicative-expression*:

cast-expression

multiplicative-expression * *cast-expression*

multiplicative-expression / *cast-expression*

multiplicative-expression % *cast-expression*

(6.5.7) *additive-expression*:

multiplicative-expression

additive-expression + *multiplicative-expression*

additive-expression - *multiplicative-expression*

(6.5.8) *shift-expression*:

additive-expression

shift-expression << *additive-expression*

shift-expression >> *additive-expression*

(6.5.9) *relational-expression*:

shift-expression

relational-expression < *shift-expression*

relational-expression > *shift-expression*

relational-expression <= *shift-expression*

relational-expression >= *shift-expression*

(6.5.10) *equality-expression*:

relational-expression
equality-expression **==** *relational-expression*
equality-expression **!=** *relational-expression*

(6.5.11) *AND-expression*:

equality-expression
AND-expression **&** *equality-expression*

(6.5.12) *exclusive-OR-expression*:

AND-expression
exclusive-OR-expression **^** *AND-expression*

(6.5.13) *inclusive-OR-expression*:

exclusive-OR-expression
inclusive-OR-expression **|** *exclusive-OR-expression*

(6.5.14) *logical-AND-expression*:

inclusive-OR-expression
logical-AND-expression **&&** *inclusive-OR-expression*

(6.5.15) *logical-OR-expression*:

logical-AND-expression
logical-OR-expression **||** *logical-AND-expression*

(6.5.16) *conditional-expression*:

logical-OR-expression
logical-OR-expression **?** *expression* **:** *conditional-expression*

(6.5.17.1) *assignment-expression*:

conditional-expression
unary-expression *assignment-operator* *assignment-expression*

(6.5.17.1) *assignment-operator*: one of

= ***=** **/=** **%=** **+=** **-=** **<<=** **>>=** **&=** **^=** **|=**

(6.5.18) *expression*:

assignment-expression
expression **,** *assignment-expression*

(6.6) *constant-expression*:

conditional-expression

A.3.2 Declarations

(6.7.1) *declaration*:

declaration-specifiers *init-declarator-list*_{opt} **;**
attribute-specifier-sequence *declaration-specifiers* *init-declarator-list* **;**
static_assert-declaration
attribute-declaration

(6.7.1) *declaration-specifiers*:

declaration-specifier *attribute-specifier-sequence*_{opt}
declaration-specifier *declaration-specifiers*

(6.7.1) *declaration-specifier*:

storage-class-specifier
type-specifier-qualifier
function-specifier

(6.7.1) *init-declarator-list*:

init-declarator
init-declarator-list **,** *init-declarator*

(6.7.1) *init-declarator*:

declarator
declarator = *initializer*

(6.7.1) *attribute-declaration*:

attribute-specifier-sequence ;

(6.7.2) *storage-class-specifier*:

auto
constexpr
extern
register
static
thread_local
typedef

(6.7.3.1) *type-specifier*:

void
char
short
int
long
float
double
signed
unsigned
_BitInt (*constant-expression*)
bool
_Complex
_Decimal32
_Decimal64
_Decimal128
atomic-type-specifier
struct-or-union-specifier
enum-specifier
typedef-name
typeof-specifier

(6.7.3.2) *struct-or-union-specifier*:

struct-or-union *attribute-specifier-sequence*_{opt} *identifier*_{opt} { *member-declaration-list* }
struct-or-union *attribute-specifier-sequence*_{opt} *identifier*

(6.7.3.2) *struct-or-union*:

struct
union

(6.7.3.2) *member-declaration-list*:

member-declaration
member-declaration-list *member-declaration*

(6.7.3.2) *member-declaration*:

*attribute-specifier-sequence*_{opt} *specifier-qualifier-list* *member-declarator-list*_{opt} ;
static_assert-declaration

(6.7.3.2) *specifier-qualifier-list*:

type-specifier-qualifier *attribute-specifier-sequence*_{opt}
type-specifier-qualifier *specifier-qualifier-list*

(6.7.3.2) *type-specifier-qualifier*:

type-specifier
type-qualifier
alignment-specifier

(6.7.3.2) *member-declarator-list*:

member-declarator
member-declarator-list , *member-declarator*

(6.7.3.2) *member-declarator*:

declarator
*declarator*_{opt} : *constant-expression*

(6.7.3.3) *enum-specifier*:

enum *attribute-specifier-sequence*_{opt} *identifier*_{opt} *enum-type-specifier*_{opt}
 { *enumerator-list* }
enum *attribute-specifier-sequence*_{opt} *identifier*_{opt} *enum-type-specifier*_{opt}
 { *enumerator-list* , }
enum *identifier* *enum-type-specifier*_{opt}

(6.7.3.3) *enumerator-list*:

enumerator
enumerator-list , *enumerator*

(6.7.3.3) *enumerator*:

enumeration-constant *attribute-specifier-sequence*_{opt}
enumeration-constant *attribute-specifier-sequence*_{opt} = *constant-expression*

(6.7.3.3) *enum-type-specifier*:

: *specifier-qualifier-list*

(6.7.3.5) *atomic-type-specifier*:

_Atomic (*type-name*)

(6.7.3.6) *typeof-specifier*:

typeof (*typeof-specifier-argument*)
typeof_unqual (*typeof-specifier-argument*)

(6.7.3.6) *typeof-specifier-argument*:

expression
type-name

(6.7.4.1) *type-qualifier*:

const
restrict
volatile
_Atomic

(6.7.5) *function-specifier*:

inline
_Noreturn

(6.7.6) *alignment-specifier*:

alignas (*type-name*)
alignas (*constant-expression*)

(6.7.7.1) *declarator*:

*pointer*_{opt} *direct-declarator*

(6.7.7.1) *direct-declarator*:

identifier *attribute-specifier-sequence*_{opt}
(*declarator*)
array-declarator *attribute-specifier-sequence*_{opt}
function-declarator *attribute-specifier-sequence*_{opt}

(6.7.7.1) *array-declarator*:

direct-declarator [*type-qualifier-list*_{opt} *assignment-expression*_{opt}]
direct-declarator [**static** *type-qualifier-list*_{opt} *assignment-expression*]
direct-declarator [*type-qualifier-list* **static** *assignment-expression*]
direct-declarator [*type-qualifier-list*_{opt} *]

- (6.7.7.1) *function-declarator*:
direct-declarator (*parameter-type-list*_{opt})
- (6.7.7.1) *pointer*:
 * *attribute-specifier-sequence*_{opt} *type-qualifier-list*_{opt}
 * *attribute-specifier-sequence*_{opt} *type-qualifier-list*_{opt} *pointer*
- (6.7.7.1) *type-qualifier-list*:
type-qualifier
type-qualifier-list *type-qualifier*
- (6.7.7.1) *parameter-type-list*:
parameter-list
parameter-list , ...
 ...
- (6.7.7.1) *parameter-list*:
parameter-declaration
parameter-list , *parameter-declaration*
- (6.7.7.1) *parameter-declaration*:
*attribute-specifier-sequence*_{opt} *declaration-specifiers* *declarator*
*attribute-specifier-sequence*_{opt} *declaration-specifiers* *abstract-declarator*_{opt}
- (6.7.8) *type-name*:
specifier-qualifier-list *abstract-declarator*_{opt}
- (6.7.8) *abstract-declarator*:
pointer
*pointer*_{opt} *direct-abstract-declarator*
- (6.7.8) *direct-abstract-declarator*:
 (*abstract-declarator*)
array-abstract-declarator *attribute-specifier-sequence*_{opt}
function-abstract-declarator *attribute-specifier-sequence*_{opt}
- (6.7.8) *array-abstract-declarator*:
*direct-abstract-declarator*_{opt} [*type-qualifier-list*_{opt} *assignment-expression*_{opt}]
*direct-abstract-declarator*_{opt} [**static** *type-qualifier-list*_{opt} *assignment-expression*]
*direct-abstract-declarator*_{opt} [*type-qualifier-list* **static** *assignment-expression*]
*direct-abstract-declarator*_{opt} [*]
- (6.7.8) *function-abstract-declarator*:
*direct-abstract-declarator*_{opt} (*parameter-type-list*_{opt})
- (6.7.9) *typedef-name*:
identifier
- (6.7.11) *braced-initializer*:
 { }
 { *initializer-list* }
 { *initializer-list* , }
- (6.7.11) *initializer*:
assignment-expression
braced-initializer
- (6.7.11) *initializer-list*:
*designation*_{opt} *initializer*
initializer-list , *designation*_{opt} *initializer*
- (6.7.11) *designation*:
designator-list =
- (6.7.11) *designator-list*:
designator
designator-list *designator*

- (6.7.11) *designator*:
 [*constant-expression*]
 . *identifier*
- (6.7.12) *static_assert-declaration*:
 static_assert (*constant-expression* , *string-literal*) ;
 static_assert (*constant-expression*) ;
- (6.7.13.2) *attribute-specifier-sequence*:
 *attribute-specifier-sequence*_{opt} *attribute-specifier*
- (6.7.13.2) *attribute-specifier*:
 [[*attribute-list*]]
- (6.7.13.2) *attribute-list*:
 *attribute*_{opt}
 attribute-list , *attribute*_{opt}
- (6.7.13.2) *attribute*:
 attribute-token *attribute-argument-clause*_{opt}
- (6.7.13.2) *attribute-token*:
 standard-attribute
 attribute-prefixed-token
- (6.7.13.2) *standard-attribute*:
 identifier
- (6.7.13.2) *attribute-prefixed-token*:
 attribute-prefix :: *identifier*
- (6.7.13.2) *attribute-prefix*:
 identifier
- (6.7.13.2) *attribute-argument-clause*:
 (*balanced-token-sequence*_{opt})
- (6.7.13.2) *balanced-token-sequence*:
 balanced-token
 balanced-token-sequence *balanced-token*
- (6.7.13.2) *balanced-token*:
 (*balanced-token-sequence*_{opt})
 [*balanced-token-sequence*_{opt}]
 { *balanced-token-sequence*_{opt} }
 any token other than a parenthesis, a bracket, or a brace

A.3.3 Statements

- (6.8.1) *statement*:
 labeled-statement
 unlabeled-statement
- (6.8.1) *unlabeled-statement*:
 expression-statement
 *attribute-specifier-sequence*_{opt} *primary-block*
 *attribute-specifier-sequence*_{opt} *jump-statement*
- (6.8.1) *primary-block*:
 compound-statement
 selection-statement
 iteration-statement
- (6.8.1) *secondary-block*:
 statement

(6.8.2) *label*:

*attribute-specifier-sequence*_{opt} *identifier* :
*attribute-specifier-sequence*_{opt} **case** *constant-expression* :
*attribute-specifier-sequence*_{opt} **default** :

(6.8.2) *labeled-statement*:

label *statement*

(6.8.3) *compound-statement*:

{ *block-item-list*_{opt} }

(6.8.3) *block-item-list*:

block-item
block-item-list *block-item*

(6.8.3) *block-item*:

declaration
unlabeled-statement
label

(6.8.4) *expression-statement*:

*expression*_{opt} ;
attribute-specifier-sequence *expression* ;

(6.8.5.1) *selection-statement*:

if (*expression*) *secondary-block*
if (*expression*) *secondary-block* **else** *secondary-block*
switch (*expression*) *secondary-block*

(6.8.6.1) *iteration-statement*:

while (*expression*) *secondary-block*
do *secondary-block* **while** (*expression*) ;
for (*expression*_{opt} ; *expression*_{opt} ; *expression*_{opt}) *secondary-block*
for (*declaration* *expression*_{opt} ; *expression*_{opt}) *secondary-block*

(6.8.7.1) *jump-statement*:

goto *identifier* ;
continue ;
break ;
return *expression*_{opt} ;

A.3.4 External definitions

(6.9.1) *translation-unit*:

external-declaration
translation-unit *external-declaration*

(6.9.1) *external-declaration*:

function-definition
declaration

(6.9.2) *function-definition*:

*attribute-specifier-sequence*_{opt} *declaration-specifiers* *declarator* *function-body*

(6.9.2) *function-body*:

compound-statement

A.4 Preprocessing directives

(6.10.1) *preprocessing-file*:

*group*_{opt}

(6.10.1) *group*:

group-part
group *group-part*

- (6.10.1) *group-part*:
if-section
control-line
text-line
non-directive
- (6.10.1) *if-section*:
if-group elif-groups_{opt} else-group_{opt} endif-line
- (6.10.1) *if-group*:
if *constant-expression new-line group_{opt}*
ifdef *identifier new-line group_{opt}*
ifndef *identifier new-line group_{opt}*
- (6.10.1) *elif-groups*:
elif-group
elif-groups elif-group
- (6.10.1) *elif-group*:
elif *constant-expression new-line group_{opt}*
elifdef *identifier new-line group_{opt}*
elifndef *identifier new-line group_{opt}*
- (6.10.1) *else-group*:
else *new-line group_{opt}*
- (6.10.1) *endif-line*:
endif *new-line*
- (6.10.1) *control-line*:
include *pp-tokens new-line*
embed *pp-tokens new-line*
define *identifier replacement-list new-line*
define *identifier lparen identifier-list_{opt}) replacement-list new-line*
define *identifier lparen ...) replacement-list new-line*
define *identifier lparen identifier-list , ...) replacement-list new-line*
undef *identifier new-line*
line *pp-tokens new-line*
error *pp-tokens_{opt} new-line*
warning *pp-tokens_{opt} new-line*
pragma *pp-tokens_{opt} new-line*
new-line
- (6.10.1) *text-line*:
pp-tokens_{opt} new-line
- (6.10.1) *non-directive*:
pp-tokens new-line
- (6.10.1) *lparen*:
a (character not immediately preceded by white space
- (6.10.1) *replacement-list*:
pp-tokens_{opt}
- (6.10.1) *pp-tokens*:
preprocessing-token
pp-tokens preprocessing-token
- (6.10.1) *new-line*:
the new-line character
- (6.10.1) *identifier-list*:
identifier
identifier-list , identifier

- (6.10.1) *pp-parameter*:
pp-parameter-name pp-parameter-clause_{opt}
- (6.10.1) *pp-parameter-name*:
pp-standard-parameter
pp-prefixed-parameter
- (6.10.1) *pp-standard-parameter*:
identifier
- (6.10.1) *pp-prefixed-parameter*:
identifier :: identifier
- (6.10.1) *pp-parameter-clause*:
(pp-balanced-token-sequence_{opt})
- (6.10.1) *pp-balanced-token-sequence*:
pp-balanced-token
pp-balanced-token-sequence pp-balanced-token
- (6.10.1) *pp-balanced-token*:
(pp-balanced-token-sequence_{opt})
[pp-balanced-token-sequence_{opt}]
{ pp-balanced-token-sequence_{opt} }
any *pp-token* other than a parenthesis, a bracket, or a brace
- (6.10.1) *embed-parameter-sequence*:
pp-parameter
embed-parameter-sequence pp-parameter
- defined-macro-expression*:
defined *identifier*
defined (*identifier*)
- h-preprocessing-token*:
any *preprocessing-token* other than >
- h-pp-tokens*:
h-preprocessing-token
h-pp-tokens h-preprocessing-token
- header-name-tokens*:
string-literal
< *h-pp-tokens* >
- has-include-expression*:
__has_include (*header-name*)
__has_include (*header-name-tokens*)
- has-embed-expression*:
__has_embed (*header-name embed-parameter-sequence_{opt}*)
__has_embed (*header-name-tokens pp-balanced-token-sequence_{opt}*)
- has-c-attribute-express*:
__has_c_attribute (*pp-tokens*)
- va-opt-replacement*:
__VA_OPT__ (*pp-tokens_{opt}*)
- (6.10.8) *standard-pragma*:
pragma STDC FP_CONTRACT *on-off-switch*
pragma STDC FENV_ACCESS *on-off-switch*
pragma STDC FENV_DEC_ROUND *dec-direction*
pragma STDC FENV_ROUND *direction*
pragma STDC CX_LIMITED_RANGE *on-off-switch*
- (6.10.8) *on-off-switch*: one of
ON **OFF** **DEFAULT**

(6.10.8) *direction*: one of

FE_DOWNWARD	FE_TONEAREST	FE_TONEARESTFROMZERO
FE_TOWARDZERO	FE_UPWARD	FE_DYNAMIC

(6.10.8) *dec-direction*: one of

FE_DEC_DOWNWARD	FE_DEC_TONEAREST	FE_DEC_TONEARESTFROMZERO
FE_DEC_TOWARDZERO	FE_DEC_UPWARD	FE_DEC_DYNAMIC

A.5 Floating-point subject sequence

A.5.1 NaN char sequence

(7.24.2.6) *n-char-sequence*:

- digit*
- nondigit*
- n-char-sequence digit*
- n-char-sequence nondigit*

A.5.2 NaN wchar_t sequence

(7.31.4.2.2) *n-wchar-sequence*:

- digit*
- nondigit*
- n-wchar-sequence digit*
- n-wchar-sequence nondigit*

A.6 Decimal floating-point subject sequence

A.6.1 NaN decimal char sequence

(7.24.2.7) *d-char-sequence*:

- digit*
- nondigit*
- d-char-sequence digit*
- d-char-sequence nondigit*

A.6.2 NaN decimal wchar_t sequence

(7.31.4.2.3) *d-wchar-sequence*:

- digit*
- nondigit*
- d-wchar-sequence digit*
- d-wchar-sequence nondigit*